INTRODUCTION TO AI

Midterm Project Report

Robot Arm Load Analyzer

An Interactive Physics + AI Tool for Safe Robot-Arm Payload Planning

---

Sley Hortes
Student ID: 411221407
November 2026

Abstract

This report presents the Robot Arm Load Analyzer, an interactive tool for determining whether a given robot-arm pose and payload is safe to hold. The system couples three independent components behind one shared interface: a from-scratch kinematics and dynamics engine (forward/inverse kinematics, Jacobian-transpose statics, and a full Recursive Newton-Euler dynamics derivation), a trained neural-network surrogate for the same torque computation, and a real-time Qt Quick 3D interface that exposes every joint, link, and payload parameter as a live, draggable control. The two torque-computation paths are interchangeable through one common evaluator interface, which makes it possible to benchmark the learned surrogate against the exact physics honestly - reporting a measured 2.8x speed-up in one operating regime and a measured slow-down in another, rather than a single favorable number. The paper covers the system's architecture and data-processing pipeline, the physics and machine-learning methods underneath it, a set of concrete engineering problems encountered and resolved during development, and the verification discipline (148 automated test cases across 14 executables) used to establish correctness throughout.

Keywords - robot kinematics and dynamics, Recursive Newton-Euler Algorithm (RNEA), neural surrogate modeling, inverse kinematics, Qt Quick 3D, human-robot interaction, payload capacity analysis.

1. Introduction and Motivation

Every joint in a robot arm is driven by a motor with a hard torque limit. Exceed that limit anywhere in the chain and the joint stalls, strains, or fails outright. The question "is this pose, with this payload, actually safe to hold?" is therefore not cosmetic - it is the single most consequential question in robot-arm operation, and answering it correctly requires solving the arm's full kinematics and dynamics at once: the pose, the payload, gravity, and - if the arm is moving - its velocity and acceleration as well.

Industrial manufacturers answer this once, at design time, with a load-moment diagram: a payload-versus-reach curve printed on the arm's spec sheet. That curve is accurate for exactly one arm, built exactly as specified. It cannot answer "what if this link were longer," "what if this joint were heavier," or "what if I picked a different pose" - each of those requires re-deriving the dynamics by hand.

The Robot Arm Load Analyzer exists to remove that barrier. It is a from-scratch physics engine, a trained AI surrogate that approximates the same physics, and an interactive Qt Quick 3D interface, wired together so that every one of those "what if" questions gets answered the instant it is asked - by dragging a slider, not by opening a textbook.



Figure 1. The full application: 3D view, sidebar, and the per-joint bottom bar, all live-updating together.

2. Project Scope and Goals

The project was built in four planned phases - kinematics, dynamics, realistic rendering, and a set of stretch goals - and then extended well past its original roadmap based on iterative feedback, arriving at a feature set that goes considerably beyond a typical classroom kinematics demo:

- A complete forward/inverse kinematics and statics/dynamics engine, written from first principles in C++ with Eigen, not a wrapper around an existing robotics library.
- A trained neural-network surrogate for the same torque calculation, with its own honest, measured speed comparison against the exact engine - not just a toggle that claims "AI mode" without evidence.
- An arm whose own topology, geometry, and mass are editable at runtime - joints can be added or removed, and every link's length and mass can be dragged live, with every safety number recomputing immediately.
- A real pick-up interaction: staging a payload's weight does nothing by itself, but closing the gripper attaches it instantly - hanging below the gripper, wherever the arm currently is - and every joint's torque reflects the added weight immediately.
- Direct 3D manipulation - every joint is a draggable handle in the 3D view itself, not only a sidebar control.
- A payload-capacity "workspace scan" that reproduces the exact concept behind a manufacturer's load-moment diagram, computed live, through both engines, with timing.

3. System Architecture

The application is deliberately split into three independent pieces that share data only through plain, primitive types - never internal C++ or ML-framework objects crossing a boundary directly:

- **The physics engine** (`core/`, C++17 + Eigen) - the single source of truth for kinematics, statics, and dynamics. Entirely free of any UI or ML dependency, and unit-tested on its own.
- **The AI surrogate** (`ml/`, trained with PyTorch, run with a small hand-written C++ inference class) - a drop-in alternative torque evaluator trained on the physics engine's own output.
- **The interactive front end** (`app/`, Qt Quick 3D / QML) - a thin controller class exposes only plain Qt types (numbers, strings, vectors, lists) to QML, so the 3D scene and the sidebar never touch a C++ Eigen matrix or a PyTorch tensor directly.

This separation is what makes the Exact/AI comparison meaningful: both engines are called through the exact same interface (a torque-evaluator function), so switching between them, or benchmarking them side by side, is a pure substitution with nothing else changing.

3.1 Runtime Data Flow

Figure 2 traces one interaction end to end: a user action in the QML frontend updates the controller's pose/payload/mode state, which is routed to whichever torque evaluator is currently selected - the exact engine or the AI surrogate - and, for payload-capacity queries, into the shared binary-search routine that calls that evaluator repeatedly. The result flows back as plain numeric data and re-renders the 3D scene, gauges, and charts.

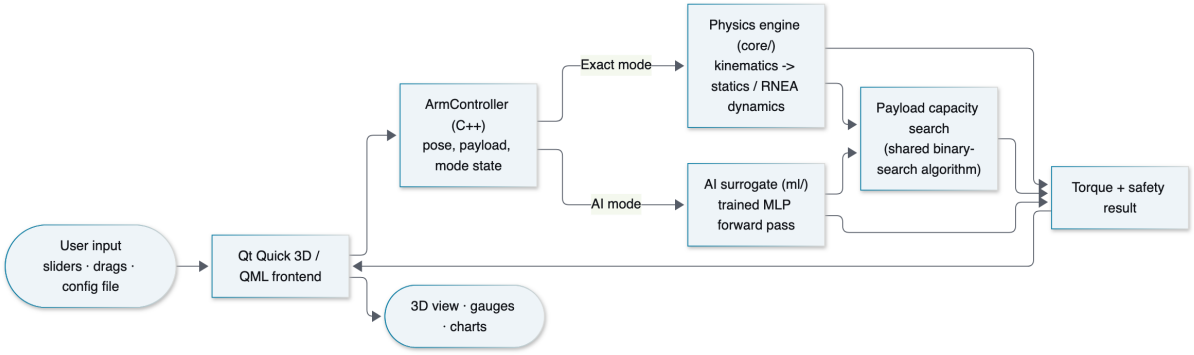


Figure 2. Runtime data-processing pipeline, from a user action to a rendered result.

3.2 AI Surrogate Training Pipeline

Figure 3 shows the offline pipeline that produces the weights the AI mode loads at runtime. Training labels are generated by calling the exact physics engine directly from C++, never re-derived in Python, so the network is always learning the project's own physics rather than a second, independently-written approximation of it.

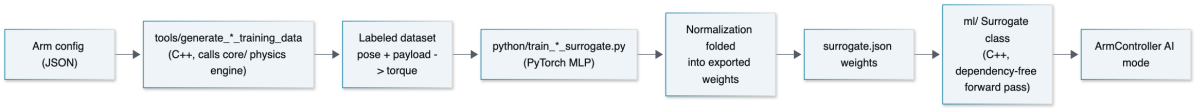


Figure 3. Offline training pipeline that produces the AI surrogate's weights.

4. The Physics Engine

4.1 Kinematics

Forward kinematics uses the classic Denavit-Hartenberg (DH) convention, chaining each joint's 4x4 homogeneous transform to place every link and the end effector in space. Inverse kinematics - "what joint angles reach this point" - is solved iteratively with damped least squares: `dq = J^T (J J^T + lambda^2 I)^-1 e`. The damping term is not a performance shortcut; it is specifically what keeps the solver numerically finite at a kinematic singularity (the arm fully extended), where a plain Jacobian pseudo-inverse would divide by a near-zero singular value and diverge.

4.2 Static Torque

At rest, joint torque is computed with the Jacobian-transpose method. This was verified independently against a virtual-work / energy-gradient derivation of the same quantity - two mathematically different routes to the same number agreeing is a far stronger correctness signal than a single derivation passing its own test.

4.3 Full Dynamics (RNEA)

When the arm is moving, static torque alone is not enough - velocity and acceleration contribute their own torque through inertia and Coriolis effects. This is computed with a full Recursive Newton-Euler Algorithm (RNEA): a forward pass propagating angular velocity, angular acceleration, and linear acceleration link by link, followed by a backward pass accumulating force and moment back to the base. The result is further decomposed into gravity, inertia, and Coriolis contributions individually, by calling the same RNEA routine three times with the other two sources zeroed each time - so the breakdown shown on screen is not an approximation of the total, it is verified to sum exactly back to it.

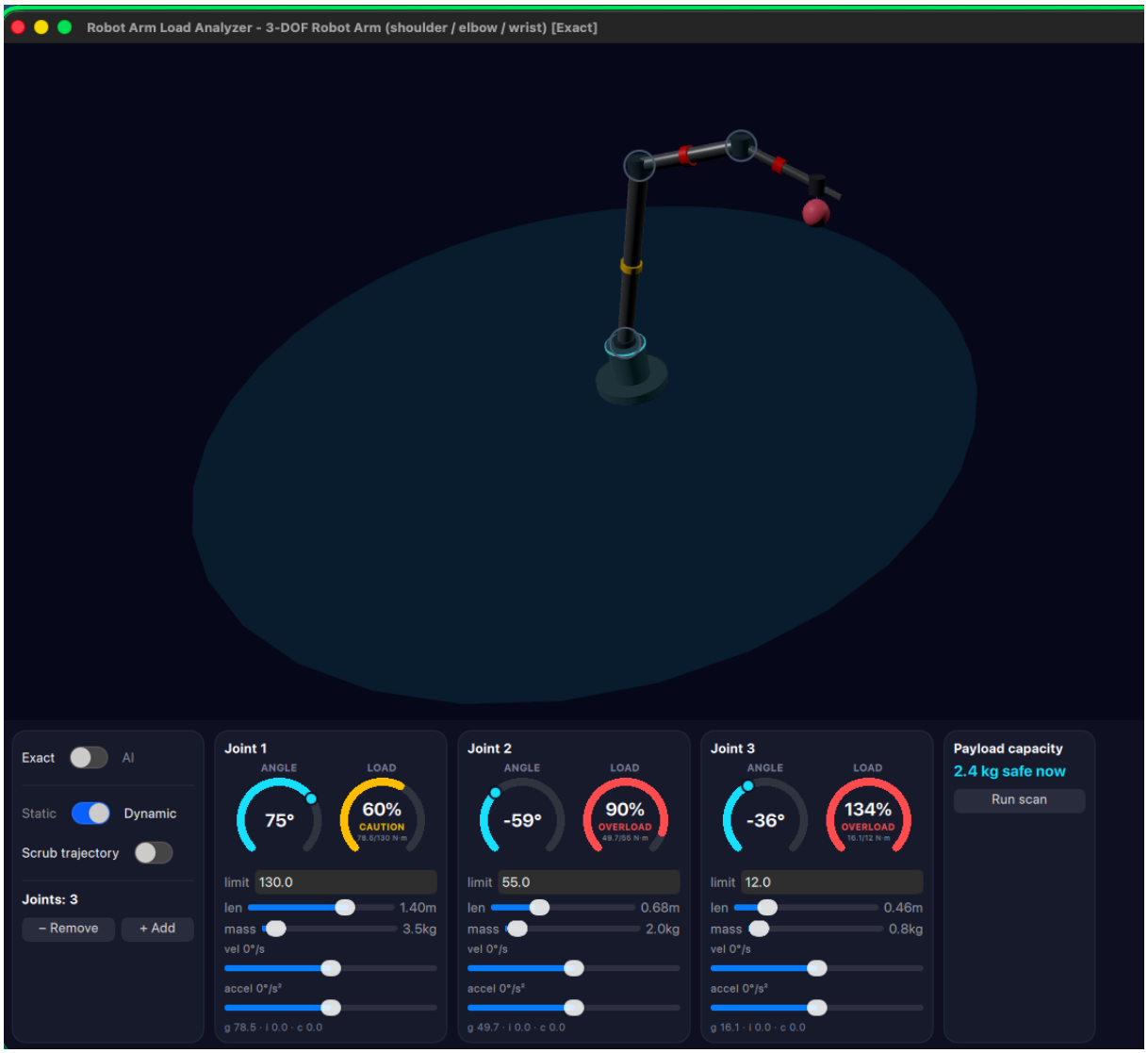


Figure 4. Dynamic mode: each joint's torque broken down into gravity (g), inertia (i), and Coriolis (c) contributions.

4.4 Payload Capacity

"How much could this arm safely lift right now" is answered by `max_safe_payload()`, a binary search over a caller-supplied torque-evaluator function. Because the evaluator is injected rather than hard-coded, the identical search algorithm drives the answer whether it is backed by the exact engine or the AI surrogate - the two are never computed by separately-written logic.

5. The AI Surrogate

5.1 Why a Surrogate

The exact engine is already fast for a single pose. The surrogate's value shows up at scale: a workspace scan evaluates dozens of candidate payloads at dozens of poses - hundreds of torque evaluations per click - and that is exactly the regime where a trained network's constant-time forward pass can out-run an iterative physics solve.

5.2 Training Data and Model

Training data is generated by a dedicated C++ tool that calls the exact physics engine directly - the training labels are never re-derived in Python, which would risk two independent (and possibly inconsistent) implementations of the same physics. The model itself is a small multilayer perceptron, trained with PyTorch, with input and output normalization folded directly into the exported weights so that the C++ inference class is a single, dependency-free forward pass - no PyTorch, no ONNX runtime, nothing but plain matrix multiplication at inference time. A separate, larger network is trained for dynamic mode, since dynamic torque is a harder function to approximate (more inputs: position, velocity, acceleration, and payload together) and needs more capacity to stay accurate.

5.3 Honest Benchmarking

Every workspace scan reports real, measured timing for both engines side by side, not an assumption. In static mode on this arm, the AI surrogate measured roughly 2.8x faster than the exact engine with curves that nearly overlap. In dynamic mode, however, the exact engine's own cost is already low enough that the network's overhead does not pay for itself, and the exact engine was measured faster instead - 0.4x, i.e. the AI path took longer. Both results are shown to the user, unedited. A tool that only ever reports the case where AI wins is not measuring anything; this one is designed to show whichever answer the numbers actually give.

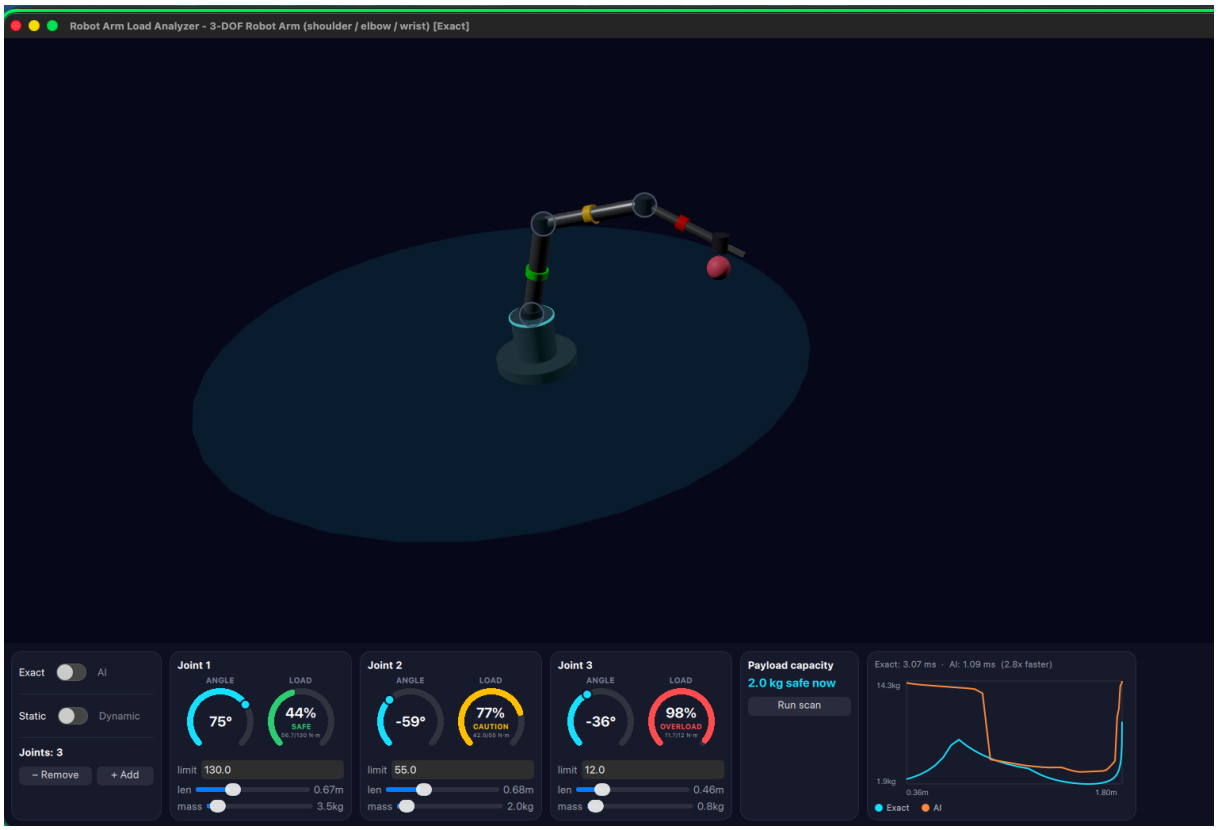


Figure 5. A workspace scan in static mode: Exact and AI curves nearly overlap, with AI measured 2.8x faster.

6. The Interactive Interface

The interface is built with Qt Quick 3D and QML. Every piece of hardware on screen - the pedestal, the limbs, the motor-drum joints - is procedural geometry (cylinders, spheres, and cubes composed and lit), not an imported 3D model, styled deliberately to read as real hardware rather than placeholder shapes. Direct manipulation is available in two forms: each joint is a draggable ring directly in the 3D view, and a flat 2D "reach pad" lets the end effector be dragged to any point in the arm's plane, with inverse kinematics solving for the joint angles behind the scenes.

Controls are split between a sidebar (reach targeting, payload, config file I/O) and a bottom bar showing every joint's own card side by side, plus the AI/Static-Dynamic mode controls and payload capacity - a layout that went through several rounds of direct feedback before settling on this split, specifically to avoid scrolling through a long, single column of controls.

7. Feature Walkthrough

- **Payload pickup.** A payload's weight is staged on a slider and has no effect until the gripper actually closes on it - closing it attaches the payload, rendered hanging below the gripper on a thin cable, and every joint's torque reflects the added weight immediately, wherever the arm happens to be posed.

- **Live geometry editing.** Dragging a joint's length or mass slider is a real change to the arm's own `ArmConfig`, not a cosmetic one - the 3D model, every torque number, and the reach envelope all recompute immediately against the new arm.
- **Payload-capacity scan.** One click sweeps the arm from fully extended to folded, computing the safe-payload curve through both engines and plotting them together with measured timing for each.
- **Dynamic mode.** Velocity and acceleration sliders appear per joint, and the torque breakdown switches from a single number to gravity/inertia/Coriolis shown separately.
- **Trajectory recording and scrubbing.** A short sequence of poses can be recorded as waypoints and then scrubbed through smoothly, with velocity and acceleration read directly off the interpolated motion rather than typed in by hand.



Figure 6. A recorded two-waypoint trajectory being scrubbed; velocity, acceleration, and the full torque breakdown update live.

8. Engineering Challenges and Solutions

Table 1 summarizes five defects encountered during development, each diagnosed to a specific root cause and verified fixed rather than patched symptomatically. They are presented here in the same problem/cause/resolution form used to document them internally at the time, since that structure is what made each one tractable to debug in the first place.

#	Issue	Root Cause	Resolution
1	**Silent sign bug in the dynamics derivation**	RNEA's forward pass used the *previous* link's angular velocity/acceleration instead of the current link's own, silently zeroing a rotating point mass's torque contribution. Two of three tests passed anyway - they never exercised the affected term.	A third, independently-designed test (pure angular acceleration vs. the parallel-axis theorem) caught it. Standing rule adopted: one passing test is never sufficient evidence for a hand-derived physics routine.
2	**Geometry bug from an inconsistent unit convention**	Qt Quick 3D primitives are modeled at a fixed 100-unit native size. Link lengths were converted to this scale; joint and payload radii were not, leaving them many meters wide next to a 1.5 m arm ("a sphere inside a cube").	One named `meshUnit` constant applied consistently to every `scale` value, never to `position`.
3	**A singularity misread as a performance bug**	The entire reach boundary is a kinematic singularity - every point there leaves an irreducible position error the damped least-squares solver can never close, so each call burned its full iteration budget, felt as input lag.	Exit as soon as the solver's own proposed step becomes negligible, not only when the position error itself is small - more iterations cannot help past that point regardless.
4	**An overcorrection that traded away a real capability**	A first fix clamped every target to 97% of true max reach to eliminate the lag above. That removed the lag but also made the arm's own genuine 100% extension permanently unreachable - a bigger behavioral change than intended.	Clamp removed once noticed; only the early-exit fix (case 3) was kept. Lesson: the most literal reading of a proposed fix is not automatically safe if it quietly trades away a capability nobody meant to give up.
5	**Rendering crash traced to its real root cause**	The app crashed on a deferred repaint triggered by clicking a tab, never on the first frame. A debugger backtrace placed the fault entirely inside the OS's native widget-styling code, not application logic.	Switching to a non-native Qt Quick Controls style ran the identical sequence cleanly, confirming a known offscreen-rendering/native-widget incompatibility - not an application defect.

Table 1. Engineering case studies: issue, diagnosed root cause, and resolution.

A pattern recurs across all five: automated checks (unit tests, qmllint, a clean launch) narrowed where to look but did not, by themselves, replace diagnosis - each fix followed from identifying the specific mechanism at fault (a wrong index, a missing conversion, a structural singularity, an over-broad clamp, a platform incompatibility), not from trial and error. Section 9 covers the verification discipline these cases motivated.

9. Testing and Verification Discipline

The project is backed by a GoogleTest suite covering every core physics module - kinematics, statics, dynamics, trajectories, inverse kinematics, payload capacity, the AI surrogate, and config file parsing - alongside the Qt-facing controller layer, for 148 individual test cases across 14 executables. That figure is deliberately re-derived by direct enumeration (`--gtest_list_tests`) rather than carried forward from memory, after an earlier, uncritically-repeated figure turned out to be wrong.

Verification is not limited to the test suite. Every QML change is checked with `qmllint` for structural issues static analysis can catch, followed by an actual timed launch of the application checked for a clean, warning-free run, and a full clean rebuild plus the complete test suite before any branch is merged. Several real defects - including the native-style rendering crash above - were only ever caught by this combination of checks, not by any single one of them alone.

10. Results and Reflections

The finished application answers its original question - "is this pose, with this payload, actually safe" - instantly and interactively, for an arm whose own shape is itself editable. It demonstrates that an AI surrogate can be meaningfully faster than a from-scratch physics engine at scale, while also demonstrating, with equal honesty, the cases where it is not. The physics underneath is independently cross-checked rather than trusted on a single derivation, and the interface was shaped by repeated rounds of direct feedback into a layout built around actual use, not a first guess.

What this project is ultimately meant to show is not just that a robot arm can be simulated, but that a simulation can be built with the same rigor expected of the system it is modeling: numbers that are verified, not assumed; comparisons that are measured, not claimed; and a design that keeps changing in response to how it actually gets used.

11. Conclusion and Future Work

The Robot Arm Load Analyzer meets and extends its original scope: a complete physics engine, a benchmarked AI surrogate, and an interactive 3D interface, integrated into one tool that makes a traditionally static engineering question - the payload-vs-reach derating curve - live and explorable. Natural extensions include training the AI surrogate to generalize across link length and mass (today it is retrained implicitly only for the geometry it was trained on), full 3D (non-planar) kinematic chains, and exporting a recorded trajectory as an animation clip for external playback.